



**[LAB] FDA 文件提取智能體：從正則到自癒 AI 的演進**

# 從一個資料工程的踩坑經驗開始

## 個人與背景資訊

- 林志忠 Data Service Manager & GenAI Strategist
- 世博科技顧問
- 智慧財產 × 法規情報
- 政府公開資料 → 可訂閱的商業情報
- 今天：一個系統失效 → 一套方法論

你的公司，現在是否也面臨這三個問題？

**PDF 格式一改**  
提取程式就壞掉

**三層式抽取 + 幾何**  
座標定位，格式不再是痛點

**需要抽取新欄位**  
要從頭重寫一套規則

**自癒 Agent 代理**  
生成並驗證提取程式

**AI 直接給答案**  
不知道答案能不能信

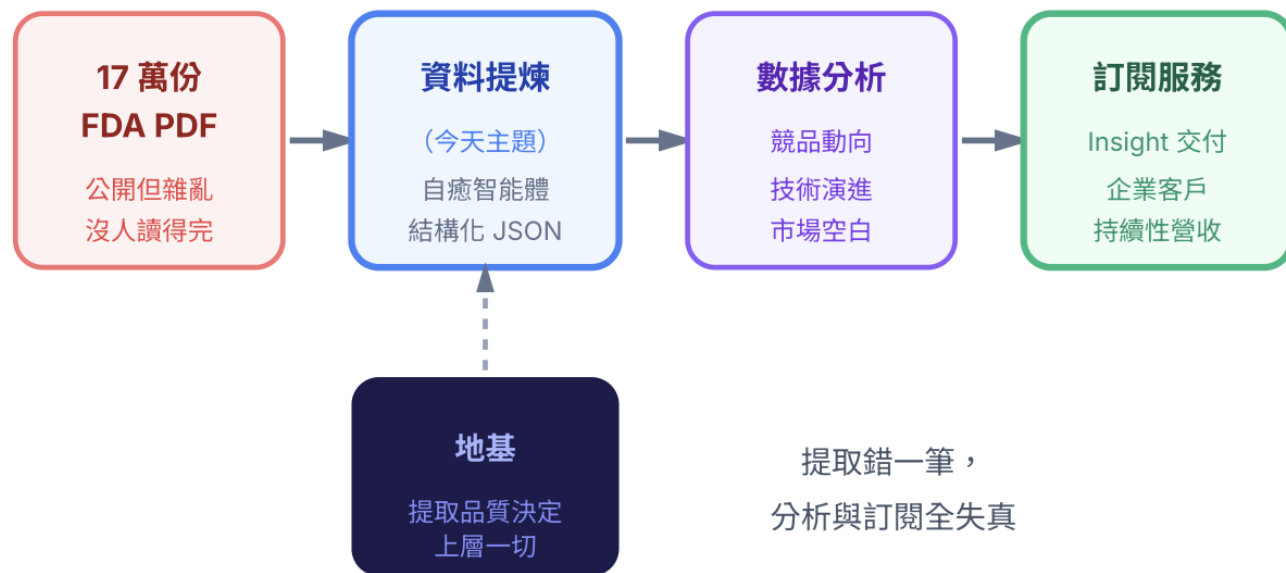
**讓 AI 寫出可跑、可測**  
可版控、自行驗證的程式

# 我們為什麼要處理 17 萬份 FDA 文件？

一個 Alert，我們重新審視整個架構

- FDA 510(k)：美國醫材進市場的許可證
- 30 年 × 17 萬份 公開，全是 PDF
- 提取失效 = 情報失真 = 訂閱服務信譽歸零
- 工程師的挑戰：格式一改，程式要怎麼不崩？
- 實際產出：<https://iximedtech.com/>

## 從原始 PDF 到訂閱營收的價值鏈



## 某一天：一次系統失效的經驗

一個空格，三天工程工時

- 提取率：95% → 23%
- 原因：K-number 前面多了一個空格
- 代價：三天工程工時
- 反思：規則永遠補不完



# 增加外掛：三層架構 + 自癒 Agent

IFU 走管線，K-number 走自癒 Agent，兩條路

- IFU 適應症：Tier 1 → Tier 2 → Tier 3 Fallback
- K-number：自癒 Agent → AI 自動產出程式碼
- 兩條路完全獨立，服務不同目的
- 換文件類型 × 換目標欄位，骨架不動

兩條提取路徑，服務不同目的



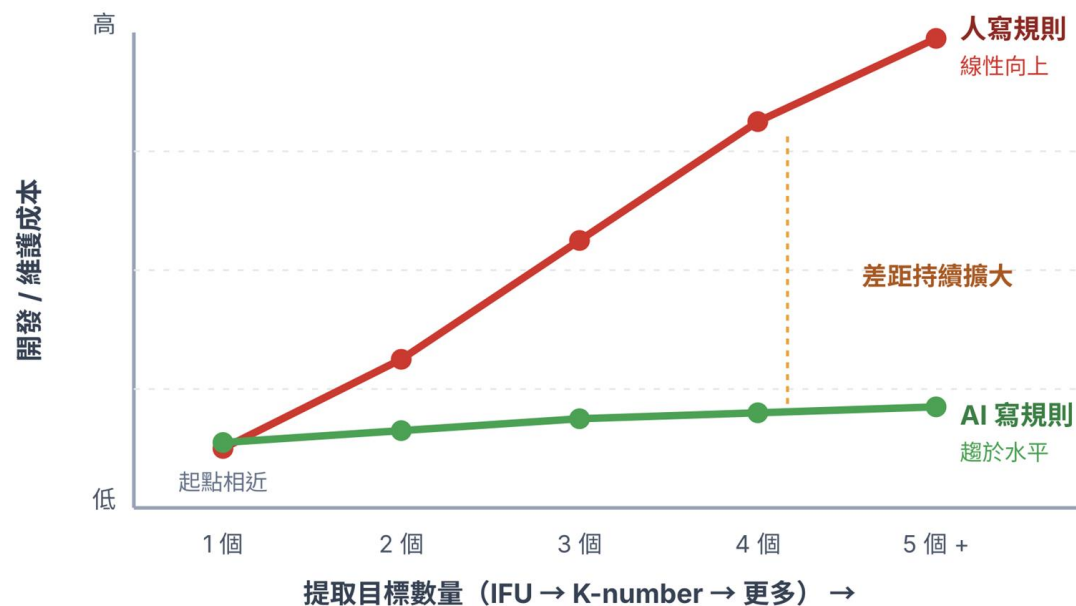


## 轉捩點：「人寫規則」到「AI 寫規則」

從「補了三十條規則」到「讓 AI 自己寫程式」

- 第一階段：IFU 適應症 → run\_rule\_processor\_xx.py
- 遭遇：數十條定錨規則，每條都是一步一腳印踩出來
- 新需求：K-number → 另一套完全不同的邏輯
- 第二階段：auto\_healing\_extractor.py → AI 自己寫程式

### 從原始 PDF 到訂閱營收的價值鏈

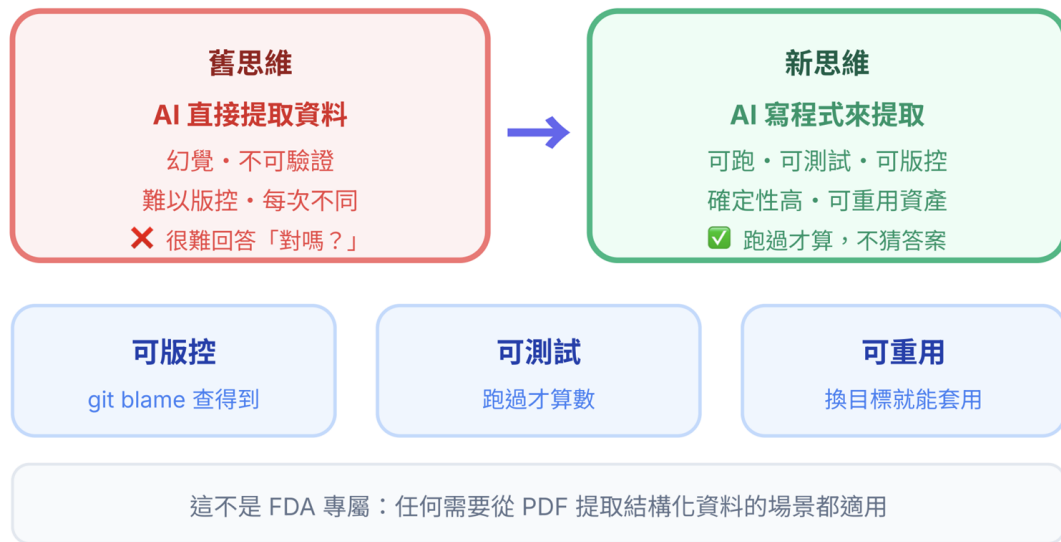


# 讓 AI 寫程式來提取資料

一個觀念的轉換，解鎖了 4 萬份文件的處理能力

- 4萬+ 份 FDA 文件，已完成結構化提取
- 舊思維：AI 直接回答 → 幻覺、不可驗證
- 新思維：AI 寫程式 → 可跑、可測試、可重用
- 腳本是持久的工程資產，不是一次性的 AI 回答

目前成效：4 萬+ 份文件已完成結構化提取



# LAB 實作前環境準備

所需時間: 10 mins



# 環境準備

- GitHub ⇒ [https://github.com/kahnlín/iTHome\\_Lab](https://github.com/kahnlín/iTHome_Lab)
- VS Code or Antigravity IDE
- LM Studio (Model:qwen2.5-coder-14b-instruct-mlx)
- Python 3.10+ ( 建議 3.14 ) — 核心執行環境

```
# 快速環境健康檢查 (Terminal 執行)
python -c "
import sys, pdfplumber, pandas as pd
print(f'Python   : {sys.version[:6]}')
print(f'pdfplumb: {pdfplumber.__version__}')
print(f'pandas   : {pd.__version__}')
print('環境基礎就緒！')
"
```

# Step 1 : pip install 安裝所需套件

LAB 需要 5 個套件，一行指令全裝好

- pdfplumber：讀取 PDF 文字、表格、座標 ( Lab 1+2 核心 )
- pandas：將提取結果輸出為 CSV
- langchain-openai：連接 LM Studio 或 GPT-4o 的 LLM 介面 ( 自癒 Agent )
- langgraph：建構 Coder / Executor / Critic 三節點 Agent 圖 ( 自癒 Agent )

```
# requirements.txt (本 LAB 所有套件)
pdfplumber>=0.10.0
pandas>=2.0.0
openai>=1.0.0
langchain-openai>=0.1.0
langgraph>=0.1.0

# — 安裝方式一：一行搞定 (推薦) —————
$ pip install -r requirements.txt
```

## Step 2：確認工作目錄結構

三支腳本 + input\_pdfs 資料夾 = 今天 LAB 的全部材料

- input\_pdfs/ 資料夾：放入 3-5 份 FDA 510(k) PDF 樣本  
( 已放上github )
- lab\_01\_regex.py：任務一的腳本
- lab\_02\_geometric.py：任務二的腳本
- auto\_healing\_extractor.py：LangGraph Agent

# 工作目錄應該長這樣：

```
./
├── input_pdfs/
│   ├── K123456_summary.pdf
│   ├── K234567_510k.pdf
│   └── ... (3-5 份即可啟動)
├── lab_01_regex.py
├── lab_02_geometric.py
├── auto_healing_extractor.py
└──
# 執行後會自動生成：
├── lab_01_regex.csv
├── final_universal_extractor.py
└── final_results.csv
```

# Phase 1 實作

所需時間: 20 mins (2個任務)

# Phase 1 實作目的：感受 Regex 的極限

先讓傳統工具告訴你問題邊界在哪

- 任務一：執行 lab\_01\_regex.py → 看初期能抓到什麼、漏掉什麼（輸出 lab\_01\_regex.csv）
- 任務二：執行 lab\_02\_geometric.py → 看進化幾何空間能補回多少（輸出 lab\_02\_geometric.csv）
- 對比兩份 CSV：N/A 的消失率
- 完成 Phase 1，你會真正理解：為什麼我們之後需要讓 AI 寫 PDF 提取程式碼

# 代碼解說：lab\_01\_regex.py ( 精準 Regex )

Stage 1 的四個 Pattern 假設 PDF 文字結構永遠規則

- K\_NUMBER\_PATTERN 用 `[Kk]\d{6}` 防止誤匹配，但 K 和數字之間不能有空格
- `precise_patterns` 對應 4 種 FDA 標準格式 ( Predicate K number、510(k)... )
- 任何一個 Pattern 都假設 PDF 裡有明確的關鍵字且位置固定——格式一亂就全部失效
- 失敗時：靜默回傳 (`[]`, `[]`)，CSV 顯示 N/A

```
# === [Stage 1] 精準 Regex 匹配 ===  
precise_patterns = [  
    (r"Predicate\s+(?:Device\s+)?K\s+number[:\s]+([Kk]\d{6})", "pred"),  
    (r"Predicate\s+510\.(k\).*?([Kk]\d{6})", "pred"),  
    (r"Reference\s+(?:Device\s+)?K\s+number[:\s]+([Kk]\d{6})", "ref"),  
    (r"predicate\s+device\s+is\s+([Kk]\d{6})", "pred")  
]
```



# 實作任務 (一)：體驗lab\_01\_regex.py ( 精準 Regex )

執行傳統 Regex，觀察目標格式多樣性的執行結果

- 目標：從 PDF 中準確抓取 K-Number (例：K123456)
- 步驟 1：開啟終端機
- 步驟 2：執行 `python lab_01_regex.py` ( 輸出：lab\_01\_regex.csv )
- 步驟 3：觀察結果 — 正常 PDF 抓到 K-Number；變種/複雜 PDF 回傳 N/A

```
(.venv) kahn_pro@Kahn-ProdeMacBook-Pro itHome_Lab % python lab_01_regex.py
📁 找到 13 個 PDF 檔案，開始批次提取...
[13/13] 正在處理：K254290.pdf ...
✅ 處理完成！
📄 結果已儲存至：lab_01_regex.csv
```

# 預期結果：lab\_01\_regex.csv 長什麼樣？

記下哪幾份 PDF 顯示 N/A

- 成功案例：文字結構標準的 PDF → Predicate Devices 顯示 K-Number ( 例：K254283 )
- 失敗案例：表格跑版 / 多欄 / OCR 錯誤的 PDF → 顯示 N/A
- 關鍵觀察：同一份 PDF，Predicate 抓到但 Reference 是 N/A？或兩個都 N/A？

| File Name   | Predicate Devices | Reference Devices |
|-------------|-------------------|-------------------|
| K254288.pdf | N/A               | N/A               |
| K254275.pdf | N/A               | N/A               |
| K250002.pdf | N/A               | N/A               |
| K254305.pdf | N/A               | N/A               |
| K254271.pdf | N/A               | N/A               |
| K250003.pdf | N/A               | N/A               |
| K254267.pdf | N/A               | N/A               |
| K250001.pdf | N/A               | N/A               |
| K254283.pdf | K130924           | N/A               |



| Subject Device<br>(TBD)     | Predicate Device<br>(K130924) |
|-----------------------------|-------------------------------|
| BruxZir® Shaded 16 PLUS     | BruxZir® Shaded               |
| Prismatik Dentalcraft, Inc. | Prismatik Dentalcraft, Inc.   |
| EIH                         | EIH                           |
| Yes                         | Yes                           |

## 實作任務 (二) : lab\_02\_geometric.py

不找文字，找空間座標——讓 K-Number 無處可躲

- 以 Predicate 標題詞的 Box 為錨點，向下 150px、左右各 80px 框出搜尋區域
- `extract_words()` 把每個詞當作獨立 Box 物件，不受文字提取順序影響
- 搜尋框內符合 `K\d{6}` 的詞就是 K-Number——空間位置比文字結構穩定得多

```
# === [Stage 10] 幾何座標精準定位 (含容錯) ===  
try:  
    words = page.extract_words(keep_blank_chars=False)  
    anchors = []  
    for word in words:  
        txt = word['text'].lower()  
        if "predicate" in txt or "primary" in txt or ("reference" in txt and "device" in txt):  
            anchors.append(word)  
  
    for anchor in anchors:  
        search_box = {  
            'x0': anchor['x0'] - 80,  
            'x1': anchor['x1'] + 80,  
            'top': anchor['bottom'],  
            'bottom': anchor['bottom'] + 150  
        }
```

# 對比兩份 CSV，看多了什麼

N/A 消失的數量 = 幾何定位的貢獻，也是 AI 需要自動生成這段代碼的理由

- 對比欄位：lab\_01\_regex.csv vs lab\_02\_geometric.csv 的 Predicate Devices
- 如果 lab\_02\_geometric.py 也找不到：可能是 PDF 格式超出 Box 150px 範圍，這就是 LangGraph Agent 的任務

lab\_01\_regex

| File Name   | Predicate Devices | Reference Devices |
|-------------|-------------------|-------------------|
| K254288.pdf | N/A               | N/A               |
| K254275.pdf | N/A               | N/A               |
| K250002.pdf | N/A               | N/A               |
| K254305.pdf | N/A               | N/A               |
| K254271.pdf | N/A               | N/A               |
| K250003.pdf | N/A               | N/A               |
| K254267.pdf | N/A               | N/A               |
| K250001.pdf | N/A               | N/A               |
| K254283.pdf | K130924           | N/A               |
| K254269.pdf | N/A               | N/A               |
| K254278.pdf | N/A               | N/A               |
| K254253.pdf | N/A               | N/A               |
| K254290.pdf | N/A               | N/A               |

lab\_02\_geometric

| File Name   | Predicate Devices                                    | Reference Devices |
|-------------|------------------------------------------------------|-------------------|
| K254288.pdf | N/A                                                  | N/A               |
| K254275.pdf | N/A                                                  | N/A               |
| K250002.pdf | K112402                                              | N/A               |
| K254305.pdf | N/A                                                  | N/A               |
| K254271.pdf | K041242                                              | N/A               |
| K250003.pdf | K192063                                              | N/A               |
| K254267.pdf | N/A                                                  | N/A               |
| K250001.pdf | N/A                                                  | N/A               |
| K254283.pdf | K130924, K220816                                     | N/A               |
| K254269.pdf | N/A                                                  | N/A               |
| K254278.pdf | K180792, K180846                                     | N/A               |
| K254253.pdf | K041860, K113652, K121495, K211582, K230141, K233008 | N/A               |
| K254290.pdf | N/A                                                  | N/A               |

# Phase 2 實作

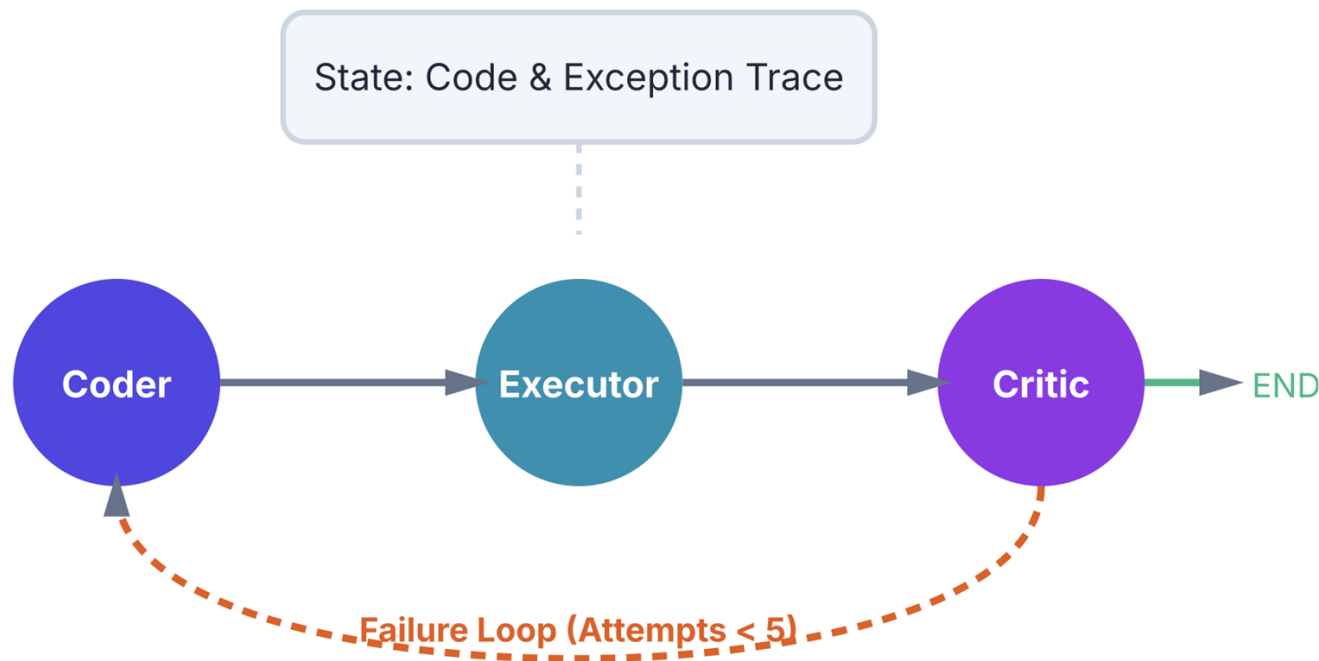
所需時間: 50 mins (自癒 Agent )

# LangGraph Agent 是整個 FDA 提取系統的核心大腦

為什麼 while loop 不夠，State 物件才是關鍵

- while loop：每次重試，上次報錯消失
- LangGraph State：error\_trace 共享
- 條件路由：Critic 通過 → END / 失敗  
→ 攜帶報錯回 Coder
- 自癒迴圈（方法論）× LangGraph（技術手段）

迭代進化：每次失敗，系統變得更聰明一點





# 代碼解說：Coder Node 長什麼樣？

error\_message x Prompt → 自癒出發點

- 前 4 次 Local LM Studio → 連線失敗自動切 GPT-4o
- Fallback → 第 5 次起 GPT-4o Expert Mode
- Prompt 設計, instruction 把 error\_message + feedback 塞入 (error\_context 變數) , 讓 AI 讀到診斷報告
- 理解這個 Prompt 是 AI 的診療單——它讀了什麼錯誤, 才能改什麼策略

```
# =====  
# 節點 1: Coder (工程師) [Prompt 同步 v11]  
# =====  
def coder_node(state: AgentState):  
    current_attempt = state['attempts'] + 1  
  
    # === 混合策略設定 ===  
    MAX_LOCAL_ATTEMPTS = 4  
  
    if current_attempt <= MAX_LOCAL_ATTEMPTS:  
        # [Tier 1] Local LLM  
        model_source = "Local LLM"  
        print(f"\n🟦 [Coder] 第 {current_attempt} 次嘗試：使用 {model_source}...")  
        try:  
            llm = ChatOpenAI(  
                base_url="http://localhost:1234/v1",  
                api_key="lm-studio",  
                model="local-model",  
                temperature=0,  
                # 🔥 增加 Timeout 時間 (解決 Request timed out)  
                timeout=300,  
                # 🔥 硬上限：防止小模型陷入重複退化、無限疊加 Stage 而生成超長代碼  
                max_tokens=4000  
            )  
        except Exception as e:  
            print(f"⚠️ Local LLM 連線失敗，切換至 GPT-4o...")  
            model_source = "GPT-4o (Fallback)"  
            llm = ChatOpenAI(model="gpt-4o", temperature=0, max_tokens=4000)
```

# 代碼解說：Critic Node：誰說提取成功了？

Critic 是確定性規則引擎，不是 AI，這是整個架構最重要的設計決策

- 核心原則：驗證絕對不能交給 AI——AI 可能說「成功了」但結果是空的或格式錯誤
- 三層驗證：① 非空檢查 → ② K-Number 格式正則 → ③ 可選白名單比對
- Critic 是系統的「誠實錨點」：不花任何 Token，卻是整個迴圈能停下來的原因
- Critic 回傳失敗時，error\_message 被填入 State，下一輪 Coder 繼續改寫

```
# --- 節點 3: Critic (評審) ---
def critic_node(state: AgentState):
    print("🟢 [Critic] 正在檢驗結果...")

    result = state.get("execution_result")
    error = state.get("error_message")

    if error:
        if "NoneType" in str(error):
            return {"feedback": "檢測到表格處理錯誤 (NoneType)。請檢查 `if cell` 防呆。", "status": "retry"}
        return {"feedback": f"程式執行發生錯誤: {error}", "status": "retry"}

    # 處理回傳格式
    predicates = []
    references = []

    if isinstance(result, dict):
        predicates = result.get("predicate", [])
        references = result.get("reference", [])
    elif isinstance(result, tuple) or isinstance(result, list):
        if len(result) >= 1: predicates = result[0]
        if len(result) >= 2: references = result[1]
    else:
        return {"feedback": "回傳格式錯誤，必須是 (predicates, references) 的 Tuple 或 Dict。", "status": "retry"}

    if not predicates and not references:
        print(" -> 檢驗失敗：提取結果為空。")
        return {
            "feedback": "提取結果為空。請檢查 Regex 或幾何範圍，但不要刪除任何現有的提取策略。",
            "status": "retry"
        }
```

# 實作任務：執行 auto\_healing\_extractor.py

## AI 如何從錯誤中學習，自動演化提取策略

- 執行指令：python auto\_healing\_extractor.py ( 在工作目錄的 Terminal 執行 )
- 熱啟動機制：Agent 先試 INITIAL\_GOLDEN\_CODE (v11)，成功就不用 LLM 訓練
- 需要訓練時：Coder→Executor→Critic 迴圈最多 5 次，前 4 次用 LM Studio，第 5 次 GPT-4o
- 訓練完成後：final\_universal\_extractor.py 自動更新 ( 這就是 AI 生成的提取代碼 )

```
(.venv) kahn_pro@Kahn-ProdeMacBook-Pro itHome_Lab % python auto_healing_extractor.py
開始進化式提取流程 (v12 穩定版)，共 13 個訓練檔案...

[1/13] 訓練檔案: K254288.pdf
嘗試使用現有最佳代碼 (v11)...
現有代碼提取為空，需要進化...

[Coder] 第 1 次嘗試：使用 Local LLM...
[Executor] 正在執行代碼...
[Critic] 正在檢驗結果...
-> 檢驗失敗：提取結果為空。

[Coder] 第 2 次嘗試：使用 Local LLM...
[Executor] 正在執行代碼...
[Critic] 正在檢驗結果...
-> 檢驗失敗：提取結果為空。

[Coder] 第 3 次嘗試：使用 Local LLM...
[Executor] 正在執行代碼...
[Critic] 正在檢驗結果...
-> 檢驗失敗：提取結果為空。
```

```
[13/13] 訓練檔案: K254290.pdf
嘗試使用現有最佳代碼 (v11)...
現有代碼提取為空，需要進化...

[Coder] 第 1 次嘗試：使用 Local LLM...
[Executor] 正在執行代碼...
[Critic] 正在檢驗結果...
-> ✓ 檢驗通過！Predicate: []
代碼進化成功！更新 Global Best Code。

=====
正在生成最終執行檔...
✓ 成功！已建立 'final_universal_extractor.py'
請執行：python final_universal_extractor.py
```

# 驗收：final\_universal\_extractor.py

AI 自癒的成果——它應該比你手寫的更全面

- 確認 final\_universal\_extractor.py 已更新
- 差距就是 AI 自動加上的新策略: 檢查全文中可能的 K Number
- 執行 python final\_universal\_extractor.py → 輸出 final\_results.csv

```
# === 新策略: 檢查全文中可能的 K Number ===  
# 這個策略會在全文中尋找所有可能的 K Number, 並根據上下文判斷其類型  
all_k_numbers = K_NUMBER_PATTERN.findall(full_text)  
for k_number in all_k_numbers:  
    k_number = k_number.upper()  
    if k_number != current_k:  
        # 確認 K Number 周圍的上下文  
        k_index = full_text.find(k_number)  
        context_window = full_text[max(0, k_index - 100):k_index + 100].lower()  
        if "predicate" in context_window:  
            predicates.add(k_number)  
        elif "reference" in context_window:  
            references.add(k_number)
```



# 對比最後二份 CSV，看多了什麼

N/A 消失的數量 = AI 自癒生成的貢獻

- 對比欄位：lab\_02\_geometric.csv vs final\_results.csv 的 Predicate Devices

lab\_02\_geometric

| File Name   | Predicate Devices                                    | Reference Devices |
|-------------|------------------------------------------------------|-------------------|
| K254288.pdf | N/A                                                  | N/A               |
| K254275.pdf | N/A                                                  | N/A               |
| K250002.pdf | K112402                                              | N/A               |
| K254305.pdf | N/A                                                  | N/A               |
| K254271.pdf | K041242                                              | N/A               |
| K250003.pdf | K192063                                              | N/A               |
| K254267.pdf | N/A                                                  | N/A               |
| K250001.pdf | N/A                                                  | N/A               |
| K254283.pdf | K130924, K220816                                     | N/A               |
| K254269.pdf | N/A                                                  | N/A               |
| K254278.pdf | K180792, K180846                                     | N/A               |
| K254253.pdf | K041860, K113652, K121495, K211582, K230141, K233008 | N/A               |
| K254290.pdf | N/A                                                  | N/A               |

final\_results

| File Name   | Predicate Devices                                             | Reference Devices |
|-------------|---------------------------------------------------------------|-------------------|
| K254288.pdf | K173767                                                       | K241815, K242436  |
| K254275.pdf | K230311                                                       | N/A               |
| K250002.pdf | K112402                                                       | N/A               |
| K254305.pdf | N/A                                                           | N/A               |
| K254271.pdf | K041242                                                       | N/A               |
| K250003.pdf | K192063                                                       | N/A               |
| K254267.pdf | N/A                                                           | N/A               |
| K250001.pdf | N/A                                                           | N/A               |
| K254283.pdf | K130924, K220816                                              | N/A               |
| K254269.pdf | N/A                                                           | N/A               |
| K254278.pdf | K180792, K180846                                              | N/A               |
| K254253.pdf | K041860, K113652, K121495, K131459, K211582, K230141, K233008 | K041860, K121495  |
| K254290.pdf | K241832                                                       | N/A               |

# 我們誠實面對的限制

自癒不是萬能，設定停損點交還給人類

- 5 次上限機制：如果 AI 在兩個錯誤之間反覆橫跳，必須強制中斷
- 人工審核：承認 AI 有搞不定的極端案例，誠實交還給工程師
- 不要讓系統假裝成功：這比拋出 Exception 更加危險

## 自癒震盪現象 (Endless Loop)



5 次上限 → 人工審核 (HITL)



# 把今天的架構帶進你的場景

Agent 框架是通用型，只有業務知識需遷移

- 不需要改動：整個 LangGraph 三節點架構 + State 設計 + Conditional Edge 邏輯
- 需要改動（只有兩個地方）：coder 的任務說明 + critic 的驗證規則
- 遷移步驟：換任務描述 → 換驗證規則 → 準備 5-10 份新文件樣本 → 執行

| 組件              | FDA 510(k)                                                   | 發票 Invoice   | 合約 Contract           |
|-----------------|--------------------------------------------------------------|--------------|-----------------------|
| 提取目標            | K-Number                                                     | 發票號碼         | 合約到期日                 |
| 定錨點特徵           | Predicate<br>標題文字                                            | 發票日期<br>欄位標籤 | 簽署方<br>欄位標籤           |
| Critic<br>驗證規則  | Kd{6}                                                        | ^INV-d{8}\$  | ^d{4}-d{2}-<br>d{2}\$ |
| Agent 框架<br>不動！ | LangGraph 三節點 + State + Conditional Edge<br>→ 完全不需要改動，直接帶走複用 |              |                       |

換場景時修改

永遠不需要動

今天你建的 Compound AI 架構，是你企業 AI 化最有價值的通用資產  
只換業務知識（2 個文件）→ 就能遷移到任何文件提取場景

# 謝謝您的聆聽！

請掃描下方連結

- 核心：讓 AI 寫程式，不是讓 AI 給答案
- AI 自癒生成程式

今天帶走一句話

**讓 AI 寫程式，  
不是讓 AI 給答案。**



[medium.com/@kahnlin](https://medium.com/@kahnlin)



[www.linkedin.com/in/kahnlin](https://www.linkedin.com/in/kahnlin)